2025 IEEE 24th International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom) | 979-8-3315-6532-9/25/$31.00 ©2025 IEEE | DOI: 10.1109/Trustcom66490.2025.00187

VLAH: A Lightweight and Verifiable Framework for Approximate Nearest Neighbor Search in High-Dimensional Space

Yiping Teng*, Yuyao Tang*, Gang Wang*†, Shiqing Wang*, Changze Li*, Liang Zhao*,
*School of Computer, Large-scale Distributed System Laboratory,
Shenyang Aerospace University, Liaoning, Shenyang, China

***Abstract*—The Approximate Nearest Neighbor (ANN) search is a cornerstone in high-dimensional data processing, which, however, remains challenging in outsourced scenarios due to potential result and data tampering. Recent studies on verifiable search have partially offered verification guarantees, while they suffer from limited scalability or fail to address result integrity simultaneously. In this paper, we propose VLAH, a lightweight and verifiable framework for ANN search. VLAH integrates locality-sensitive hashing with a compressed trie structure for efficient bucket verification and employs the in-bucket HNSW index to accelerate candidate retrieval. To ensure result integrity, we design a hybrid verification object that combines Merkle-based prefix proofs, neighbor-bucket consistency, and graph search trace validation. By integrating blockchain technology, we further enhance the integrity of the scheme. We theoretically analyze the complexity of our scheme and the correctness and integrity guarantees of the query results in lightweight verification. Experimental results on real high-dimensional datasets show that VLAH achieves competitive search performance with significantly reduced verification costs, bridging the gap between efficiency and verifiability in large-scale ANN systems.**

***Index Terms*—ANN search, Query verification, Blockchain, High-dimensional space**

I. Introduction

A. Background and Motivation

Approximate Nearest Neighbor (ANN) search plays a pivotal role in high-dimensional data analysis, with wide applications in multimedia retrieval [1], computer vision [2], and machine learning acceleration [3]. As the dimensionality and volume of data continue to grow, exact kNN methods become computationally intractable due to the curse of dimensionality. Consequently, efficient ANN techniques have been widely adopted to strike a balance between speed and accuracy.

To improve scalability, data owners often outsource ANN query services to third-party servers. However, such outsourcing raises new concerns, particularly when the service provider is untrusted. Adversaries may forge or drop query results [4], [5]. This motivates the need for ANN search mechanisms that are not only efficient and scalable but also verifiable.

Corresponding author: wanggang5338@163.com

Recent studies on verifiable search techniques [4]–[7] provide verification mechanisms for the completeness and correctness of search results in outsourced scenarios. However, these techniques are often limited to low-dimensional spaces. In high-dimensional ANN search, existing authenticated search schemes incur prohibitive computation or fail to offer correctness guarantees, while most verifiable methods lack scalability [8]. This reveals a critical gap that few unified solutions simultaneously ensure computational efficiency and result integrity for high-dimensional ANN queries.

B. Challenges and Contributions

The construction of verification object (VO) for ANN search in high-dimensional spaces is computationally prohibitive, rendering the verifiable ANN problem particularly challenging. Effective index structures that simultaneously facilitate query processing and enable rigorous result verification are therefore indispensable. However, indexing high-dimensional data introduces a constellation of difficulties. Conventional tree-based indices succumb to the curse of dimensionality, incurring exponential storage growth and degraded query fidelity [9]. Although graph-based techniques provide superior search accuracy and representational flexibility, they incur prohibitively high computational overhead during VO generation and user-side verification of query results [8]. Locality-Sensitive Hashing (LSH) schemes, while dimension-robust, exhibit acute sensitivity to hyperparameter tuning, precipitating volatile and often unpredictable performance profiles [1], [10]. The verifiable ANN search in high dimensions mainly poses the following challenges:

Challenge I: Verifiable Index Design. Efficiently indexing high-dimensional vectors without sacrificing search accuracy and supporting verification, particularly under restricted computational models, remains a significant challenge.

Challenge II: Result Verification Guarantees. Ensuring the soundness and correctness of ANN results requires a lightweight architecture and efficient validation mechanisms.

Challenge III: High-dimensional Adaptivity. Balancing search performance, verification overhead, and index quality is difficult as dimensionality and dataset size increase.

2324-9013/25/$31.00 ©2025 IEEE
DOI 10.1109/Trustcom66490.2025.00187

Authorized licensed use limited to: Chang'an University. Downloaded on April 22,2026 at 05:12:42 UTC from IEEE Xplore. Restrictions apply.

To address these challenges, we propose VLAH, a verifiable lightweight hashing framework for ANN search. We map high-dimensional vectors into LSH Morton codes. Points sharing a prefix of a prescribed length are grouped into a single hash bucket, of which each is assigned a unique bucket key. To efficiently manage the buckets, we employ a compressed trie at the upper layer to index the bucket keys. To accelerate query processing while simultaneously improving accuracy, we further construct an HNSW index over the high-dimensional vectors residing in each hash bucket. To guarantee result integrity, we equip the two-tier index with a tailored verification layer. An ADS based on Merkle trees is instantiated for both the compressed trie and the HNSW index. To further shrink the VO size and reduce the verification cost, we design a minimal covering subtree for the compressed trie and a Merkle multiproof for the bucket-level HNSW graphs. VLAH seamlessly integrates LSH, trie-based compact bucketing, and graph-assisted verification, thereby achieving scalability while guaranteeing the integrity of query results. Empirical evaluation demonstrates that VLAH reduces the size of the verification object (VO) by 30–50% and accelerates the performance of client-side verification by 3–5$\times$ compared to existing studies. The main contributions are summarized as follows.

- We present a verifiable lightweight hashing framework (VLAH) for verifiable ANN search in high-dimensional space.
- We develop an efficient ANN search method that effectively balances search performance and verification overhead in high-dimensional space.
- We construct a hybrid proof object and propose an efficient blockchain-assisted client-side verification method for high-throughput result verification.
- We theoretically analyze the complexity and security of our schemes, and conduct extensive experiments on real datasets to demonstrate the search performance and verification overhead.

The rest of the paper is organized as follows. Section II introduces related works. Section III states the problem definition and preliminaries of our study. Section IV and Section V presents our proposed scheme. Section VI presents the complexity and security analysis of our scheme. Section VII reports the experimental results and summarizes the findings of the experiments. Section VIII concludes the paper.

II. Related Work

In this section, we first review ANN indices and then examine verifiable schemes for Boolean, vector, and spatial queries. We further point out their shared limits. This sets the stage for our unified design.

Hashing-based approaches. Tao et al. [1] proposed the LSB-tree, combining locality-sensitive hashing with B-trees. R2LSH [11] improves I/O efficiency by projecting data into 2D spaces with B^+-trees. DET-LSH [12] employs a dynamic encoding tree to accelerate indexing and querying. DB-LSH 2.0 [13] further improves query performance via adaptive and query-centric bucket construction. These works offer sublinear query complexity with theoretical guarantees.

Proximity Graph. Graph-based methods have gained attention for their high accuracy. Malkov et al. [2] introduced the Navigable Small World (NSW) graph, while HNSW [14] adds a multi-layer skip-list-like hierarchy to improve scalability. EFANNA [3] combines randomized trees with kNN graphs for fast initialization. FANNG [15] exploits local low-dimensional manifolds to construct GPU-efficient graphs.

Hybrid Index. Hybrid approaches seek the best of both. MQH [16] enhances the point-to-hyperplane search by utilizing multi-level quantization. LSH-APG [17] integrates LSH-guided insertion into APG construction, balancing efficiency and accuracy.

Hash-based methods offer fast indexing and strong guarantees, while graph-based methods provide high recall and efficiency. Recent hybrid frameworks bridge these methods, aiming for scalable, accurate, and updatable ANN solutions, but are still inadequate for verification requirements.

Verifiable Boolean and Keyword Queries on Blockchain. vChain [7] supports Boolean ranges via accumulator ADS, but global re-computation after each update hurts dynamic workloads. Zhang et al. [18] authenticate keyword search with Merkle inverted indexes and Bloom filters, whose false positives and dimension blow-up enlarge proofs.

Verifiable High-Dimensional ANN Search. ANNProof [8] embeds HNSW in Merkle trees to shrink VO, yet the design is tightly coupled to HNSW and resists adaptation. ImageProof [6] secures BoVW pipelines with Merkle k-d trees and cuckoo filters, but BoVW assumptions restrict generality, and dynamic inserts trigger costly rebuilds.

Verifiable Location-based kNN. VMR-Tree [5] combines spatial neighbor indexing with Merkle hashing, yet the proof size still scales with neighborhood density, and the scheme ignores high-dimensional vectors. PAMT [19] offers proofs of dynamic consistency, but authentication paths lengthen with tree height, becoming prohibitive for large ANN datasets. For road networks, Jing et al. [20] certify road-network kNN with Voronoi diagrams. Still, the diagram must be rebuilt for every graph update and cannot handle high-dimensional vectors. SVkNN [4] adds privacy-preserving indexes, yet their cryptographic cost grows linearly with data size, limiting the web-scale deployment.

Overall, prior work sacrifices at least one of accuracy, scalability, or lightweight verification. Existing schemes for verifiable ANN search suffer from high rebuilding costs under updates, exponential degradation in high dimensions, rapidly growing proof sizes and cryptographic overheads, and tight coupling to specific index structures or data models, leaving a critical gap for a scalable, dimension-robust, and general-purpose solution.

In light of these limitations, our work seeks to design a lightweight and verifiable ANN framework that supports large-scale vector search with minimized verification overhead. By integrating efficient indexing, compact ADS construction, and

Authorized licensed use limited to: Chang'an University. Downloaded on April 22,2026 at 05:12:42 UTC from IEEE Xplore. Restrictions apply.

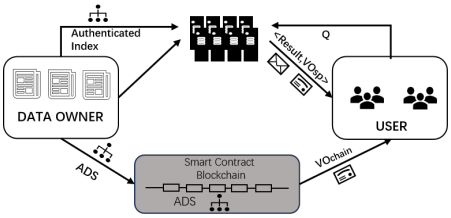


Fig. 1: System Model

modular proof composition, we aim to strike a balance between query accuracy, security, and system performance.

III. PROBLEM DEFINITION AND PRELIMINARIES

A. Problem Definition

Let $\mathcal{D} = \{p_0, p_1, \ldots, p_{n-1}\}$ be a dataset in $\mathbb{R}^d$, where each point $p_i = (p_{i1}, p_{i2}, \ldots, p_{id})$ is a d-dimensional vector. Let q be a query point, k a positive integer, and R the result set. Let ϵ denote an approximation factor, $c > 1$ an approximation ratio, $\mathcal{H}$ a locality-sensitive hashing family, $\mathcal{T}$ a Merkle Hash Tree, $\mathcal{V}$ a verification algorithm, and H_p the authentication path of p.

Definition 1: (ϵ, k)-Approximate Nearest Neighbor Query. Given a query point q, an (ϵ, k)-approximate nearest neighbor query returns k points $o_1, o_2, \ldots, o_k$ sorted in ascending order of their distances to q. For $1 \leq i \leq k$, o_i is the i-th nearest neighbor of q in $\mathcal{D}$, satisfying $\|q, o_i\| \leq (1+\epsilon) \cdot \|q, o_i^*\|$, where o_i^* is the actual i-th nearest neighbor of q.

Definition 2: Verifiable Approximate Nearest Neighbor (VANN) Query. Given a dataset $\mathcal{D}$ and a query point q, the goal is to return a top-k result set $R \subseteq \mathcal{D}$ such that

$$\max_{r \in R} d(r, q) \leq d(p, q), \quad \forall p \in \mathcal{D} - R,$$

guarantees the correctness and soundness of R.

Definition 3: Correctness of VANN Query. Given a dataset $\mathcal{D}$, a query point q, and an approximation ratio ϵ, a VANN query returns a result set R that is accepted if:

- $R \subseteq \mathcal{D}$;
- For all $p \in R$, $\|p - q\| \leq (1 + \epsilon) \cdot \min_{p' \in \mathcal{D}} \|p' - q\|$.

B. System Overview

1) System Model: As shown in Fig. 1, the system involves four parties: a data owner (DO), a cloud service provider (SP), a blockchain, and a user. Each data object is represented as $o_i = \langle id, \vec{x}_i \rangle$, where id is the identifier and $\vec{x}_i$ is the vector.

The DO sends the dataset and ADS to the SP and stores the root hash of the data on the blockchain to ensure integrity with minimal on-chain storage. The SP uses ADS to answer user queries and returns both the result and a verification object (VO). A supplementary proof VO_{chain} is taken from the blockchain. The user verifies the correctness of the result by combining VO and VO_{chain} using the DO's hash function.

2) Threat Model: In our model, it is assumed that DO, the blockchain, and query clients are trusted, while due to various issues such as program failures, security vulnerabilities, and commercial interests, SP, as a third-party service provider, may tamper with the data and therefore be treated as an untrusted party. Hence, SP needs to prove that its query results meet the following two conditions:

- **Soundness.** All objects in the result have not been tampered with and originated from DO.
- **Correctness.** All results satisfy the query conditions.

C. Cryptography Primitives

SHA-256 Hash Algorithm. SHA-256 is a widely used cryptographic hash function standardized by the National Institute of Standards and Technology (NIST) as part of the SHA-2 family [21]. SHA-256 satisfies two cryptographic properties:

- **One-way property:** Given a message m, it is computationally efficient to compute $h(m)$, but infeasible to recover m from $h(m)$.
- **Collision resistance:** It is hard to find any two distinct inputs $m_1 \neq m_2$ such that $h(m_1) = h(m_2)$.

Merkle Hash Tree [22]. The Merkle Hash Tree (MHT) is constructed bottom-up by first hashing all leaf nodes derived from the original data. Each internal node is computed by hashing the concatenation of its children's hashes (e.g., $H_1 = h(H_3 \| H_4)$, where "$\|$" denotes string concatenation).

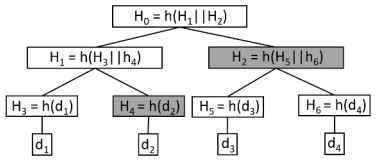


Fig. 2: Merkle Hash Tree

D. Locality Sensitive Hash and Radix Tree

Locality Sensitive Hashing (LSH). LSH is a technique used for similarity search in high-dimensional data. It maps similar data points to the same hash bucket with high probability. The hash function for the Euclidean distance is given by [1]:

$$h(o) = \left\lfloor \frac{h^*(o) + b}{w} \right\rfloor, \tag{1}$$

where w is a pre-defined integer and b is a real number chosen uniformly from $[0, w)$. To distinguish these two LSH families, we call h^* the projected function and h the hash function. This hash function is (d_1, d_2, p_1, p_2)-sensitive, meaning that the probability of two points being hashed to the same bucket is p_1 if their distance is less than d_1, and p_2 if their distance is greater than d_2.

Radix Tree. The *Radix Tree*, or *compressed prefix tree*, is a space-efficient structure for storing string keys with common prefixes. It merges chains of single-child nodes into edges labeled with substrings, reducing memory usage and improving

Authorized licensed use limited to: Chang'an University. Downloaded on April 22,2026 at 05:12:42 UTC from IEEE Xplore. Restrictions apply.

cache performance. Variants like Patricia Trie are applied in ADS, especially in blockchain systems where compactness and verifiability are essential [23], [24].

IV. Hash Trie-based Authenticated Hybrid Index

A. Basic ADS Structure

To enable efficient and verifiable approximate nearest neighbor (ANN) search, we propose a Hash Trie-based Authenticated Hybrid Index. This index is designed to combine high retrieval efficiency with end-to-end integrity verification. It achieves this by integrating locality-sensitive hashing (LSH), hierarchical in-bucket search, and cryptographic authentication across the full search path. Specifically, VLAH comprises four synergistic components: (1) an LSH-based Morton code bucketing scheme for locality-aware partitioning, (2) a compressed prefix trie over bucket keys to support efficient routing and verification, (3) an HNSW graph within each bucket for fast approximate search, and (4) a two-layer authenticated data structure (ADS) for both global trie traversal and local result validation. This layered architecture not only supports scalable search but also ensures cryptographic verifiability of the returned results.

In the first stage, each data point $\vec{x}_i$ is transformed via a locality-sensitive hash function into a bitstring:

$$h_i = \text{LSH}(\vec{x}_i) \in \{0,1\}^d \tag{2}$$

This bitstring is further encoded into a sortable Morton code:

$$m_i = \text{Morton}(h_i) \tag{3}$$

All data points are sorted by m_i and evenly partitioned into fixed-size buckets B_i. Each bucket is assigned a *bucket key* k_i, typically chosen as the prefix of the first Morton code in the bucket, and maintains a continuous code range.

To organize and authenticate the bucket-level structure globally, the set of keys $\{k_i\}$ is indexed using a compressed prefix trie $\mathcal{T}$. Each leaf node in the trie corresponds to a bucket and stores its Merkle root R_i, which is calculated from bucket contents. Internal nodes recursively aggregate the hashes of their child nodes using:

$$H(v) = H(p_v \| H(c_1) \| H(c_2) \| \cdots \| H(c_k)) \tag{4}$$

where p_v is the prefix at node v, and $c_1, \ldots, c_k$ are its children. The root hash R_{trie} of $\mathcal{T}$ serves as the global commitment of the full index structure, allowing for efficient verifiable routing.

Within each bucket B_i, we construct a Merkle-based authenticated data structure over its HNSW graph $G_i = (V_i, E_i)$. For each node $x \in V_i$, we compute the following hashes:

- **Vector hash:** $h_x^v = H(\vec{x})$
- **Adjacency hash:** $h_x^a = H(\text{Adj}(x))$, where $\text{Adj}(x) = \{(\text{id}_j, \delta(x,j))\}$
- **Leaf hash:** $L_x = H(\text{id}_x \| h_x^v \| h_x^a)$

The authenticated data structure is constructed as follows:

1) Compute L_x for each node $x \in V_i$.

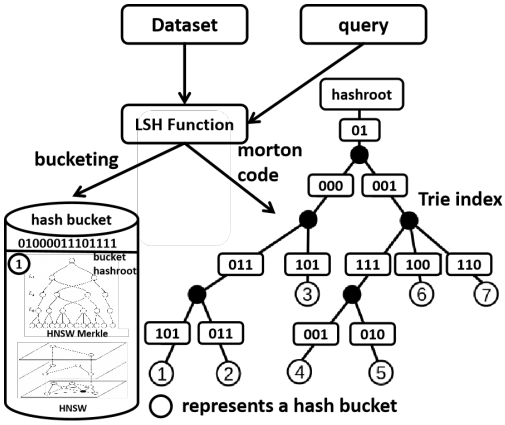


Fig. 3: VLAH Structure

2) Build a binary Merkle tree over all $\{L_x\}$, and record the root hash R_i as the Merkle commitment for bucket B_i.
3) Store the position of each L_x in the Merkle tree to support efficient multiproof generation.

This structure allows the user to verify to the existence and integrity of returned nodes, as well as verify the correctness of their neighbor relationships within the HNSW graph.

The ADS in VLAH is hierarchically organized, comprising a global *Merkle Trie* for verifying bucket-level routing and a local *Merkle HNSW* structure for authenticating in-bucket approximate results. This layered design supports end-to-end integrity verification without requiring access to the full dataset. To enhance verification granularity, optional per-level HNSW commitments may be incorporated, albeit with increased proof overhead.

B. Optimization

Although the original ADS of VLAH supports end-to-end integrity, its verification object (VO) may become large due to redundant Merkle paths in both the global trie and local buckets. Each accessed bucket introduces an independent proof, and multiple result nodes in the same bucket require separate Merkle paths, increasing both the VO size and the client verification cost.

To address these limitations, we propose an optimized ADS design that aggregates shared paths and reduces redundant components. At the global level, we construct a *minimal covering subtree* in the prefix trie to jointly verify multiple buckets, reducing path proof complexity from $\mathcal{O}(n \log T)$ to $\mathcal{O}(\log T + n)$. At the bucket level, we adopt **Merkle multiproof** to batch-verify multiple nodes, reducing the per-bucket proof size from $\mathcal{O}(k \log N)$ to $\mathcal{O}(\log N + k)$.

Additionally, we introduce an optional **access-path digest** to compress the verification of HNSW traversal traces. These enhancements yield a more compact and efficient ADS while preserving the layered verifiability of the original framework.

Example Consider four 3D points with IDs 1–4: $p_1 = (1.2, 3.4, 0.8)$, $p_2 = (1.0, 3.5, 0.7)$, $p_3 = (9.1, 2.4, 5.5)$, $p_4 =$

Authorized licensed use limited to: Chang'an University. Downloaded on April 22,2026 at 05:12:42 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1: Optimized ADS Construction on VLAH

Input: Point set $\mathcal{D} = \{(\mathrm{id}_i, \vec{x}_i)\}$
Output: Global ADS Merkle root R_{trie}

foreach $(id_i, \vec{x}_i) \in \mathcal{D}$ **do**
  $h_i \leftarrow \mathrm{LSH}(\vec{x}_i)$; $m_i \leftarrow \mathrm{Morton}(h_i)$;
Sort $\mathcal{D}$ by m_i ascending; Partition into buckets $\{B_1, \ldots, B_T\}$;
foreach *bucket* B_i **do**
  Derive bucket key k_i from prefix of first m_i in B_i;
/* Construct bucket-level ADS with multiproof support */
foreach *bucket* $B_i = \{(id_x, \vec{x}, \mathcal{N}(x))\}$ **do**
  foreach *node* $x \in B_i$ **do**
    $h_x^v \leftarrow H(\vec{x})$; Sort $\mathcal{N}(x)$ by id;
    $h_x^a \leftarrow H(\mathcal{N}(x))$;
    $L_x \leftarrow H(\mathrm{id}_x \| h_x^v \| h_x^a)$;
  Build Merkle tree over $\{L_x\}$; Let root be R_i; Enable multiproof support;
/* Construct global trie ADS with prefix aggregation */
Insert all (k_i, R_i) into compressed prefix trie $\mathcal{T}$;
Aggregate minimal covering subtree $\mathcal{P}_{\text{merged}}$ from $\mathcal{T}$;
foreach *internal node* $v \in \mathcal{P}_{merged}$ **do**
  $H(v) \leftarrow H(p_v \| H(c_1) \| \ldots \| H(c_k))$;
/* Commit access-path digest */
foreach *bucket* B_i **do**
  $\psi_i \leftarrow \mathrm{TraceDigest}(G_i)$
return $R_{trie} \leftarrow MerkleRoot(\mathcal{T})$

$(8.9, 2.6, 5.6)$. After computing LSH and Morton codes, the dataset is sorted and split into two buckets: $B_0 = \{p_1, p_2\}$ with key $k_0 = \texttt{0110}$, and $B_1 = \{p_3, p_4\}$ with key $k_1 = \texttt{1011}$. For verification-ready bucket ADS, each node x in B_i carries its local HNSW adjacency $\mathrm{Adj}(x)$; we compute $h_v(x) = H(\vec{x})$, $h_a(x) = H(\mathrm{Adj}(x))$, and the leaf commitment $L_x = H(\mathrm{id}_x \| h_v(x) \| h_a(x))$. A Merkle tree is built over $\{L_x\}_{x \in B_i}$ to obtain the bucket root R_i, and the leaf positions are recorded to support multiproofs. At the global layer, we insert (k_0, R_0) and (k_1, R_1) into a compressed prefix trie T; each internal node hash is $H(v) = H(p_v \| H(c_1) \| \cdots \| H(c_k))$, yielding the committed root R_{trie}. To reduce future VO size, shared trie prefixes of $\{k_0, k_1\}$ are aggregated into a minimal covering subtree and only per-bucket delta paths are retained; in each bucket, storing leaf positions enables batching multiple L_x in one Merkle multiproof. Optionally, we precompute an access-path digest $\psi_i = \mathrm{TraceDigest}(G_i)$ for the in-bucket HNSW graph to summarize jump traces. Finally, the data owner publishes R_{trie} (and optionally $\{R_i\}$) to a public registry or blockchain as an external commitment. This completes index construction together with all verification commitments needed for later queries in a single build pipeline.

Leaf commitment and bucket ADS $(\mathrm{L}_x, R_i, multiproofpositions)$ :: $contentReference[oaicite : 0]index = 0$: $contentReference[oaicite : 1]index = 1Compressedtriehashingandglobalroot R_trie$ :: $contentReference[oaicite : 2]index = 2Minimalcoveringsubtree$ + $Merklemultiproofoptimization$ :: $contentReference[oaicite : 3]index = 3Access - pathdigestTraceDigest(G_i)$ :: $contentReference[oaicite : 4]index = 4Externalanchoringonblockchain$ :: $contentReference[oaicite : 5]index = 5$

V. Verifiable ANN Search Processing Scheme

A. Verifiable ANN Search Processing and VO Generation

Algorithm 2: Verifiable ANN Query in VLAH

Input: $\vec{q}$: query vector, K: number of neighbors
Output: $\mathcal{R}_q$: result set, VO_q: verification object

Compute $h_q = \mathrm{LSH}(\vec{q})$; $m_q = \mathrm{Morton}(h_q)$
Extract prefix $k_q = \mathrm{Prefix}(m_q)$
Traverse trie using k_q, record main_trie_path to B_{main}
Retrieve Merkle root R_{main} of B_{main}
Select neighbor buckets $\{B_1, \ldots, B_k\}$ via Morton order or prefix proximity
Merge trie path and compute minimal covering subtree; store per-bucket deltas in neighbor_bucket_proofs
Initialize global candidate heap $C_q \leftarrow \varnothing$
foreach *bucket* $B_j \in \{B_{main}, B_1, \ldots, B_k\}$ **do**
  Load HNSW graph $G_j = (V_j, E_j)$ from B_j
  Initialize priority queue $Q \leftarrow \{(x_0, \delta(\vec{q}, \vec{x}_0))\}$
  for $\ell = 0$ *to* L **do**
    while $Q \neq \varnothing$ *and access* $\ll M_j$ **do**
      Pop (x, d_x) from Q
      foreach $y \in \mathcal{N}(x)$ **do**
        Compute $\delta(\vec{q}, \vec{y})$
        Insert (y, d_y) to Q if among top-M_t
        Insert (y, d_y) to C_q
        Record id_y, $\vec{y}$, $\mathrm{Adj}(y)$
        Compute $L_y = H(\mathrm{id}_y \| H(\vec{y}) \| H(\mathrm{Adj}(y)))$
        Store multiproof leaf position of L_y in B_j
Record jump traces or compute digest $\psi_j \leftarrow \mathrm{TraceDigest}(G_j)$ (optional)
Sort C_q by distance; extract Top-K result set $\mathcal{R}_q$
foreach $y \in \mathcal{R}_q$ **do**
  Retain id_y, $\vec{y}$, $\mathrm{Adj}(y)$, bucket ID, multiproof position
Aggregate leaf positions per bucket; compute bucket-level multiproofs $\{\pi_j\}$
Deduplicate π_y and merge overlapping jump nodes
Optionally compress $\mathrm{Adj}(y)$ using hash summaries
Assemble VO_q: { main_trie_path, neighbor_bucket_proofs, $\mathcal{R}_q$, $\{\pi_j\}$, ψ_j, $\{R_j\}$ }
return $\mathcal{R}_q$, VO_q

We describe the optimized verifiable ANN search algorithm, which enhances the original VLAH framework by reducing proof redundancy and enabling parallelized VO generation. The process begins by computing the query's LSH code h_q and Morton code m_q, followed by prefix extraction k_q to locate the main bucket B_{main} (Line 1–2). The trie is traversed accordingly, and the minimal set of node digests is recorded as main_trie_path (Line 3), ensuring compact and deterministic global routing proofs.

To improve accuracy while balancing cost, the algorithm selects a small number of neighboring buckets $\{B_1, \ldots, B_k\}$

Authorized licensed use limited to: Chang'an University. Downloaded on April 22,2026 at 05:12:42 UTC from IEEE Xplore. Restrictions apply.

via Morton or prefix proximity (Line 5). Their trie paths are aggregated by extracting shared prefixes, and only bucket-specific suffixes are stored in neighbor_bucket_proofs (Line 6), significantly reducing redundant Merkle material.

Each target bucket B_j is independently searched using its HNSW graph G_j (Line 8–20). A bounded priority queue is used per bucket (Line 10), with access depth limited by M_j (Line 12). For each visited node y, the algorithm collects $(\mathrm{id}_y, \vec{y}, \mathrm{Adj}(y))$, computes the leaf hash $L_y = H(\mathrm{id}_y \| H(\vec{y}) \| H(\mathrm{Adj}(y)))$, and stores its Merkle position for multiproof generation (Line 18–20).

After accumulating candidates in the global heap C_q (Line 7, 16), the algorithm ranks all nodes by distance and selects the top-K results $\mathcal{R}_q$ (Line 22). To minimize VO size, overlapping Merkle paths $\{\pi_y\}$ are merged (Line 25), and jump traces are compacted. Furthermore, each $\mathrm{Adj}(y)$ may be truncated or replaced with a hash digest (Line 26), enabling size-accuracy trade-offs.

All bucket-level searches are executed in parallel, each generating a local VO fragment. These are merged into a unified VO_q with shared structures deduplicated (Line 27), reducing both generation latency and client verification cost.

Overall, the algorithm integrates prefix compression, bucket multiproofs, adjacency summarization, and thread-level parallelism to deliver accurate and verifiable top-K results with minimal overhead.

B. Client Verification

Given the returned result $\mathcal{R}_q$ and its verification object VO_q, the client first extracts the main trie path, neighbor bucket proofs, result node data, access trace, and Merkle roots (Line 1). It additionally retrieves external commitments, such as trusted trie and bucket roots, from a public registry or blockchain anchor (Line 2).

It then recomputes h_q, m_q, and the prefix k_q, and verifies that the main trie path correctly routes to the expected main bucket B_{main}. The recomputed root is compared with the external trie commitment to prevent tampering or rollback (Line 3–4). For all neighbor buckets, Merkle roots are reconstructed from the shared prefix tree and delta proofs (Line 5–7). Each reconstructed root must match the corresponding registered root to ensure the bucket content is authentic. Each result node $y \in \mathcal{R}_q$ is then validated by recomputing its leaf commitment

$$L_y = H(\mathrm{id}_y \| H(\vec{y}) \| H(\mathrm{Adj}(y))),$$

and verifying that its Merkle multiproof π_y leads to a valid root among those committed (Line 8–13). If an access-path digest ψ is provided, it is recomputed from the full trace and checked for consistency; otherwise, the client replays the traversal to validate the HNSW jump trace (Line 14–16). When applicable, each reconstructed bucket root R_j is also compared with externally committed values to detect forgery or outdated data (Line 17–18).

The verification succeeds only if all integrity checks pass, ensuring both the correctness of the top-K result set and the consistency of its provenance (Line 19).

Algorithm 3: Client Verification

Input: $\vec{q}$: query vector, K: expected result size, $(\mathcal{R}_q, VO_q)$: result and proof
Output: Valid $\in \{\texttt{True}, \texttt{False}\}$

1 Extract from VO_q: main_trie_path, neighbor_bucket_proofs, $\{(y, \pi_y)\}$, $\{R_j\}$, ψ, Adj summary type
2 **Fetch** `VO_chain` from blockchain: $R_{\text{trie}}^{\texttt{chain}}$, $\{R_j^{\texttt{chain}}\}$
3 Compute $h_q = \mathrm{LSH}(\vec{q})$; $m_q = \mathrm{Morton}(h_q)$; $k_q = \mathrm{Prefix}(m_q)$
4 Verify main_trie_path corresponds to k_q and $R_{\text{trie}}^{\texttt{recon}} == R_{\text{trie}}^{\texttt{chain}}$
5 **if** *mismatch* **then**
  return `False` ; `// Invalid routing path or fake trie root`
6 **foreach** *bucket proof* π_j *in neighbor_bucket_proofs* **do**
7 Reconstruct R_j from shared prefix + delta path
8 **if** $R_j \neq R_j^{chain}$ **then**
  return `False` ; `// Tampered bucket root`
9 **foreach** $(y, \pi_y) \in \mathcal{R}_q$ **do**
10 Verify well-formed: id_y, $\vec{y}$, $\mathrm{Adj}(y)$ (or digest)
11 $h_y^v \leftarrow H(\vec{y})$; $h_y^a \leftarrow H(\mathrm{Adj}(y))$
12 $L_y \leftarrow H(\mathrm{id}_y \| h_y^v \| h_y^a)$
13 Verify Merkle multiproof π_y leads to valid R_j
14 **if** *fail or* $R_j \notin \{R_j^{chain}\}$ **then**
  return `False`
15 **if** *access-path digest* ψ *used* **then**
16 Recompute or validate ψ for full trace
17 **if** *digest mismatch* **then**
  return `False`
else
18 Validate jump trace consistency from access_path
19 **return** `True` ; `// All checks passed`

Example. Consider a dataset containing 10^6 128-dimensional image feature vectors (e.g., SIFT descriptors). Each vector is encoded into a 64-bit Morton code, with the first 20 bits used as the bucket key. Each bucket stores about 20,000 vectors and is indexed by an HNSW graph. Suppose a query requests the top-k approximate nearest neighbors with $k = 3$.

The query vector $\vec{q}$ is first mapped by LSH to a 64-bit binary code, e.g.,

$$h_q = 1101011000110010\ldots$$

which is then converted to a Morton code m_q. The first 20 bits are extracted as the bucket key:

$$k_q = 11010110001100101100.$$

Using the compressed prefix trie, the server identifies the main bucket B_{main} corresponding to k_q and selects two adjacent buckets B_2 and B_3 according to Morton order for recall improvement.

An in-bucket HNSW search is performed within B_{main}, B_2, and B_3. For example, B_{main} returns $(id = 101, dist = 0.12)$ and $(id = 305, dist = 0.15)$, while B_2 returns $(id = 678, dist = 0.14)$. The candidates are merged:

$$[(101, 0.12),\ (678, 0.14),\ (305, 0.15)]$$

Authorized licensed use limited to: Chang'an University. Downloaded on April 22,2026 at 05:12:42 UTC from IEEE Xplore. Restrictions apply.

and ranked by distance to select the final result set:

$$R_q = [101, 678, 305].$$

The server then generates the verification object VO_q, which contains: (i) the main trie path proving $k_q \to B_{\text{main}}$; (ii) delta prefix proofs for neighbor buckets; (iii) Merkle multiproof paths for each result point to prove existence and neighbor list integrity; and (iv) an optional HNSW access path digest. Finally, (R_q, VO_q) is returned to the client.

Upon receiving (R_q, VO_q), the client recomputes h_q, m_q, and k_q, verifies the main trie path against the blockchain-anchored global root $R_{\text{chain_trie}}$, reconstructs and checks neighbor bucket roots R_j, and for each $y \in R_q$ verifies:

$$h_y^v = H(\vec{y}), \quad h_y^a = H(\text{Adj}(y)), \quad L_y = H(id_y \parallel h_y^v \parallel h_y^a)$$

via the provided multiproof. If an access path digest ψ is present, it is recomputed and compared; otherwise, the HNSW traversal is replayed for consistency. The client accepts R_q only if all verifications succeed.

VI. Analysis

A. Computational Complexity

Index Construction. Constructing VLAH involves three main steps. First, each data point is transformed by an LSH function and encoded into a Morton code. This transformation incurs $O(n \cdot d)$ time, where n is the number of data points and d is the dimension. Sorting these codes and partitioning them into buckets requires $O(n \log n)$ time. For each bucket, we construct an HNSW graph, which requires $O(n \log n)$ in practice for high-recall configurations. Finally, a compressed prefix trie is built over the bucket keys, with complexity proportional to $O(n \cdot \ell)$, where ℓ is the average prefix length.

Verifiable Query Processing. Given a query vector, the hash and Morton code computation take $O(d)$. Trie traversal to locate the main bucket costs $O(\log T)$, where T is the number of buckets. Searching the HNSW graphs in k' candidate buckets consumes $O(k' \cdot \log N)$, where N is the average number of nodes per bucket. VO generation includes Merkle multiproof assembly and optional trace summarization, with cost $O(\log N + k)$ per bucket. Overall, the query time scales sub-linearly with n.

Client-Side Verification. Verification involves reconstructing the global trie path, verifying the Merkle roots of involved buckets, and checking multiproofs for each result. The trie path reconstruction requires $O(\log T)$. Multiproof verification for k results in m buckets costs $O(m \cdot \log N + k)$. If jump trace digest is used, hash comparisons are constant-time; otherwise, full trace replay incurs additional $O(k \cdot d)$. Hence, the total verification time remains efficient.

B. Security Analysis

We follow formal models of verifiable queries [25], [26] and prove soundness and completeness of the VLAH framework through the following lemmas and theorems.

Lemma 1 (Soundness)**.** *VLAH is unforgeable.*

Proof. Each result point $y \in \mathcal{R}_q$ is associated with a leaf commitment $L_y = H(\text{id}_y \| H(\vec{y}) \| H(\text{Adj}(y)))$ stored in a Merkle tree. Any modification to y or its adjacency list changes L_y and thus the bucket Merkle root R_j. Since the global prefix trie aggregates all R_j, the tampered root cannot match the blockchain-anchored commitment R_{trie} due to the collision-resistance of SHA-256. Therefore, any altered or forged result will fail verification, proving unforgeability. □

Lemma 2 (Correctness)**.** *The returned result set contains all valid top-k approximate nearest neighbors within $(1+\epsilon)$ distance.*

Proof. Assume by contradiction that a valid point p within the $(1+\epsilon)$-bound is missing from $\mathcal{R}_q$. By the construction of the HNSW graphs and the inclusion of neighboring buckets, all candidates within the search radius are considered. Additionally, client-side distance reordering ensures that the top-k points from VO are correctly selected. Hence, any omitted point would have appeared in the final top-k set if it satisfied the distance bound. Contradiction proves correctness. □

Theorem 1. *Algorithm 2 can produce a verifiable and correct approximate nearest neighbor result set.*

Proof. Algorithm 2 ensures that all accessed buckets are authenticated via prefix trie paths and Merkle roots. Each returned point is verified through its leaf hash and adjacency proof. Lemma 1 ensures that tampered results are detectable, while Lemma 2 guarantees no valid neighbor is omitted. Therefore, the output is both correct and verifiable. □

Theorem 2. *Algorithm 3 can efficiently validate both the integrity and completeness of the result set.*

Proof. Client verification includes: (1) reconstructing the trie path and comparing the global root; (2) verifying bucket roots via Merkle proofs; (3) checking each L_y against VO multiproofs; (4) Validating the traversal trace or its digest. Each check relies on cryptographic primitives with proven properties. If all verifications pass, the result is accepted. If any fail, integrity or completeness is violated. Therefore, the algorithm is both sound and correct. □

VII. Experiment Evaluation

A. Experimental Setup

Setup. We implemented all the aforementioned algorithms in Python 3.12 and C++ on a Workstation with a 20-core Intel(R) Xeon(R) Gold 5218R 2.10GHz CPU and 128 GB RAM running Windows 11. We assumed that the data transmission time was not considered in our experiments, and the search response time was reported as the total calculation time averaged over 20 experiments. We primarily evaluated three indicators: VO generation time, VO size, and Client verification time.

Datasets. We compare our method with VLAH, Naive, and ANNProof, across four datasets: **audio** (dim=192, N=54387),

Authorized licensed use limited to: Chang'an University. Downloaded on April 22,2026 at 05:12:42 UTC from IEEE Xplore. Restrictions apply.

deep1M(dim=96), **gist1M** (dim=960), and **sift1M** (dim=128). We cut the last 20 bits of the LSH value as the bucket key. For the `audio` dataset, each bucket is configured to contain 1,000–2,000 data points; the HNSW index within every bucket is parameterized with $\texttt{efc} = 50$, $\texttt{efs} = 10$, and $M = 8$. For million-scale datasets, each hash bucket is set to a capacity of 20,000–50,000 data points, and the corresponding HNSW index is configured with $\texttt{efc} = 100$, $\texttt{efs} = 50$, and $M = 8$.

B. Experimental Evaluations

Performance Evaluation on VO Generation Time. As shown in Fig. 4, all methods exhibit increasing VO generation time with larger k and dataset sizes. The Naive method consistently performs the worst, demonstrating steep time growth due to its reliance on inefficient high-dimensional operations. VLAH generally achieves the best performance, with the lowest and most stable VO generation time across all datasets. ANNProof performs moderately, outperforming Naive but lagging behind VLAH. Specifically, on the `audio` dataset (Fig. 4a), VLAH maintains a sub-50ms time even at $k = 50$, while Naive exceeds 70ms. On `deep1M` (Fig. 4b), VLAH stays below 500ms, whereas Naive surpasses 1200ms. Similar trends are observed for `gist` (Fig. 4c) and `sift` (Fig. 4d), where VLAH remains efficient even at the largest scales. The occasional spikes in VLAH at higher k values are attributed to increased candidate verification overhead.

Performance Evaluation on VO Size. As shown in Fig. 5, the size of the VO increases with larger k for all methods, reflecting the need to include more candidates and auxiliary information for verification. The Naive approach consistently incurs the largest VO size, often exceeding 1,000 KB even on the relatively small `audio` dataset (Fig. 5b). The VLAH method reduces the VO size by approximately 30–50% compared to Naive, yet it still suffers from the overhead of maintaining redundant index structures. VLAH achieves the smallest and most stable VO size across all datasets. For instance, on `deep1M` (Fig. 5b), VLAH keeps the VO below 8,000 KB at $k = 50$, whereas Naive surpasses 8,000 KB. Similar trends are observed on `gist` (Fig. 5c) and `sift` (Fig. 5d), where VLAH maintains compact VOs even as the candidate set scales. The slight increase in VO size for VLAH at higher k is attributed to the inclusion of additional verification paths and encrypted digests required for correctness guarantees.

Performance Evaluation on Verification Time. As illustrated in Fig. 6, the verification time grows with larger k for all schemes, since more candidates and auxiliary proofs must be processed. The Naive approach incurs the highest cost, exceeding 70 ms on `audio` (Fig. 6a) and 350 ms on `deep1M` (Fig. 6b) when $k = 50$, owing to heavy public-key operations on every candidate. ANNProof reduces the overhead by 30–50 % but still lags behind VLAH, which consistently attains the lowest verification time across all settings: below 60 ms on `audio`, 300 ms on `deep1M`, 350 ms on `gist` (Fig. 6c), and 400 ms on `sift` (Fig. 6d) at $k = 50$. The marginal increase in VLAH's time at higher k stems from additional modular exponentiations required to verify larger candidate sets.

VLAH organizes data more efficiently, resulting in significantly fewer intermediate nodes than ANNProof and Naive. In high-dimensional spaces, tree-based indexes suffer from the curse of dimensionality, leading to data duplication across regions. This causes larger verification objects and longer verification time compared to VLAH.

VIII. Conclusion

This paper presents a lightweight and verifiable ANN search framework (VLAH) for high-dimensional data, which uses LSH and a compressed prefix trie for global bucket indexing, HNSW graphs inside buckets for fast and high-recall retrieval, and a hybrid verification layer, i.e., Merkle proofs, neighbor-bucket checks, and traversal traces, to guarantee integrity, all anchored by an immutable blockchain commitment. Theoretical analysis shows that VLAH achieves a lightweight index construction, while both querying and verification retain sublinear overhead. Experiments on real-world datasets show that VLAH reduces the size of the verification object (VO) by 30–50% and accelerates the performance of client-side verification by 3–5× compared to existing studies, thus achieving scalability and verifiability in outsourced ANN services.

ACKNOWLEDGMENT

The work was partially supported by Natural Science Foundation of Liaoning Province (2025-MS-126), the Young and Middle-aged Science and Technology Innovation Talent Support Plan of Shenyang (RC230832), and the Scientific Research Project of the Education Department of Liaoning Province (JYTMS20230272). Gang Wang is the corresponding author.

References

[1] Y. Tao, K. Yi, C. Sheng, and P. Kalnis, "Quality and efficiency in high dimensional nearest neighbor search," in *SIGMOD Conference*. ACM, 2009, pp. 563–576.

[2] Y. Malkov, A. Ponomarenko, A. Logvinov, and V. Krylov, "Approximate nearest neighbor algorithm based on navigable small world graphs," *Information Systems*, vol. 45, pp. 61–68, 2014.

[3] C. Fu and D. Cai, "Efanna: An extremely fast approximate nearest neighbor search algorithm based on knn graph," in *IEEE Transactions on Knowledge and Data Engineering*. IEEE, 2016, arXiv:1609.07228.

[4] N. Cui, X. Yang, B. Wang, J. Li, and G. Wang, "Svknn: Efficient secure and verifiable k-nearest neighbor query on the cloud platform," in *2020 IEEE 36th International Conference on Data Engineering (ICDE)*, 2020, pp. 253–264.

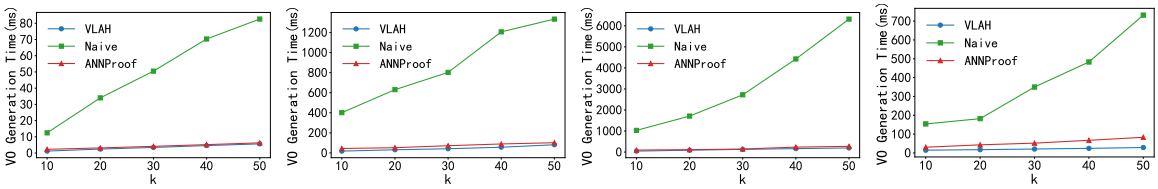
[5] Y. Teng, L. Liu, J. Qi, H. Pan, and C. Fan, "Vmr-tree: Efficient and verifiable location-based knn queries on blockchain," in *Submission to TrustCom*, 2024.

[6] S. Guo, J. Xu, C. Zhang, C. Xu, and T. Xiang, "Imageproof: Enabling authentication for large-scale image retrieval," in *2019 IEEE 35th International Conference on Data Engineering (ICDE)*, 2019, pp. 1070–1081.

[7] C. Xu, C. Zhang, and J. Xu, "vchain: Enabling verifiable boolean range queries over blockchain databases," in *Proceedings of the 2019 International Conference on Management of Data (SIGMOD)*, 2019, pp. 141–158.

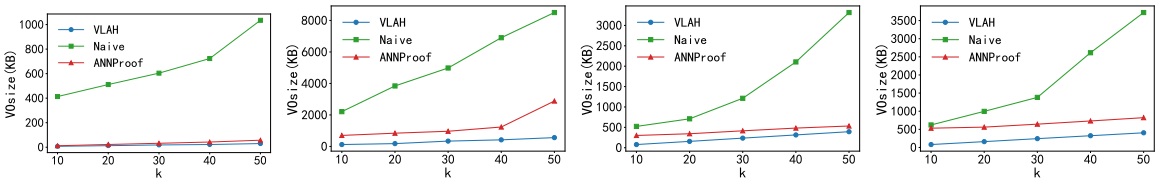
[8] L. Lu, Z. Wen, Y. Yuan, Q. He, J. Chen, and Z. Liu, "Annproof: Building a verifiable and efficient outsourced approximate nearest neighbor search system on blockchain," *Future Generation Computer Systems*, vol. 156, pp. 206–220, 2024.

Authorized licensed use limited to: Chang'an University. Downloaded on April 22,2026 at 05:12:42 UTC from IEEE Xplore. Restrictions apply.



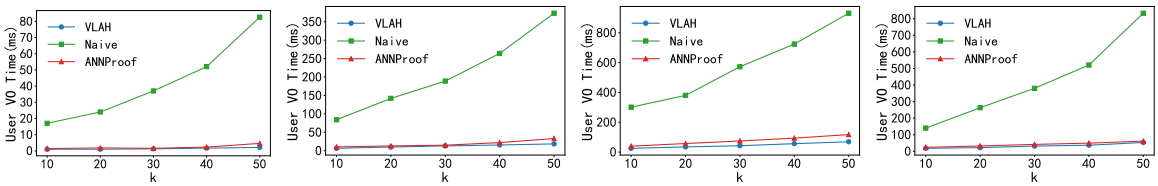
(a) audio (b) deep1M (c) gist (d) sift

Fig. 4: Performance evaluation on VO generation time vs k



(a) audio (b) deep1M (c) gist (d) sift

Fig. 5: Performance evaluation on VO size vs k



(a) audio (b) deep1M (c) gist (d) sift

Fig. 6: Performance evaluation on verification time vs k

[9] P. Indyk and R. Motwani, "Approximate nearest neighbors: Towards removing the curse of dimensionality," in *STOC*. ACM, 1998, pp. 604–613.

[10] Y. Tian, X. Zhao, and X. Zhou, "DB-LSH 2.0: Locality-sensitive hashing with query-based dynamic bucketing," *IEEE Trans. Knowl. Data Eng.*, vol. 36, no. 3, pp. 1000–1015, 2024.

[11] K. Lu and M. Kudo, "R2lsh: A nearest neighbor search scheme based on two-dimensional projected spaces," in *2020 IEEE 36th International Conference on Data Engineering (ICDE)*. IEEE, 2020, pp. 1045–1056.

[12] J. Wei, B. Peng, X. Lee, and T. Palpanas, "Det-lsh: A locality-sensitive hashing scheme with dynamic encoding tree for approximate nearest neighbor search," *Proceedings of the VLDB Endowment*, vol. 17, no. 9, pp. 2241–2254, 2024.

[13] Y. Tian, X. Zhao, and X. Zhou, "Db-lsh 2.0: Locality-sensitive hashing with query-based dynamic bucketing," *IEEE Transactions on Knowledge and Data Engineering*, vol. 36, no. 3, pp. 1000–1013, 2024.

[14] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836, 2020.

[15] B. Harwood and T. Drummond, "Fanng: Fast approximate nearest neighbour graphs," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 5713–5722.

[16] K. Lu, Y. Ishikawa, and C. Xiao, "Mqh: Locality sensitive hashing on multi-level quantization errors for point-to-hyperplane distances," in *Proceedings of the VLDB Endowment*, vol. 16, no. 4, 2022, pp. 864–876.

[17] X. Zhao, Y. Tian, K. Huang, B. Zheng, and X. Zhou, "Towards efficient index construction and approximate nearest neighbor search in high-dimensional spaces," *Proceedings of the VLDB Endowment*, vol. 16, no. 8, pp. 1979–1991, 2023.

[18] C. Zhang, C. Xu, H. Wang, J. Xu, and B. Choi, "Authenticated keyword search in scalable hybrid-storage blockchains," in *2021 IEEE 37th International Conference on Data Engineering (ICDE)*, 2021, pp. 996–1007.

[19] J. Mao, Y. Zhang, P. Li, T. Li, Q. Wu, and J. Liu, "A position-aware merkle tree for dynamic cloud data integrity verification," *Soft Computing*, vol. 21, no. 8, pp. 2151–2164, 2017.

[20] Y. Jing, L. Hu, W. Ku, and C. Shahabi, "Authentication of k nearest neighbor query on road networks," *IEEE Trans. Knowl. Data Eng.*, vol. 26, no. 6, pp. 1494–1506, 2014.

[21] National Institute of Standards and Technology, "FIPS PUB 180-4: Secure Hash Standard (SHS)," https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.180-4.pdf, 2015.

[22] R. C. Merkle, "A certified digital signature," in *Advances in cryptology—CRYPTO'89 proceedings*. Springer, 2001, pp. 218–238.

[23] H. S. de Ocáriz Borde, "An overview of trees in blockchain technology: Merkle trees and merkle patricia tries," University of Oxford Report, Tech. Rep., 2022.

[24] Y. Deng, M. Yan, and B. Tang, "Accelerating merkle patricia trie with gpu," *Proceedings of the VLDB Endowment*, vol. 17, no. 8, pp. 1856–1869, 2024.

[25] C. Zhang, C. Xu, J. Xu, and B. Choi, "Distributed knn query authentication," in *2018 19th IEEE International Conference on Mobile Data Management (MDM)*, 2018, pp. 167–176.

[26] H. Rong, H. Wang, J. Liu, W. Wu, and M. Xian, "Efficient integrity verification of secure outsourced knn computation in cloud environments," in *2016 IEEE Trustcom/BigDataSE/ISPA, Tianjin, China, August 23-26, 2016*. IEEE, 2016, pp. 236–243.

Authorized licensed use limited to: Chang'an University. Downloaded on April 22,2026 at 05:12:42 UTC from IEEE Xplore. Restrictions apply.